

Netplan & nmcli Cheat Sheet

Make network settings permanent so they survive a reboot. Changes made with **ip** are lost at reboot; these are not.

Who manages the network?

System	Config lives in	System	Config lives in
Ubuntu Server	Netplan YAML → systemd-networkd	RHEL / Fedora	NetworkManager (nmcli)
Ubuntu Desktop	Netplan YAML → NetworkManager	Debian / Proxmox	/etc/network/interfaces

Netplan: bridge main with eno1 as its port

```
network:
  version: 2
  renderer: networkd ← NetworkManager on desktops
  ethernets:
    eno1: {dhcp4: false} ← the port gets no IP
  bridges:
    main:
      interfaces: [eno1] ← plug eno1 in
      addresses: [192.168.1.50/24]
      routes:
        - to: default ← replaces old gateway4:
          via: 192.168.1.1
      nameservers:
        addresses: [192.168.1.1, 1.1.1.1]
      parameters: {stp: false} ← port up at once
```

- **File:** **/etc/netplan/01-main.yaml** (any name ending **.yaml**; read in name order).
- **DHCP instead:** replace **addresses**, **routes** and **nameservers** with **dhcp4: true**.
- **Spaces only, never tabs.** Each level is 2 spaces; a wrong indent silently moves a setting.
- **"Permissions are too open"** warning: **sudo chmod 600 /etc/netplan/*.yaml**
- **Cloud images:** cloud-init may rewrite **50-cloud-init.yaml**. Stop it with a file in **/etc/cloud/cloud.cfg.d/** saying **network: {config: disabled}**

Netplan commands (all need sudo)

Command	Does	Command	Does
netplan try	Applies, but undoes it after 120 s unless you press Enter. Use over SSH	netplan get	Merged config from all files
netplan apply	Applies for real, no safety net	netplan status	Live IPs, routes, DNS (Ubuntu 24.04)
netplan generate	Syntax check only, applies nothing	--debug	Add to any of these to see details

nmcli (NetworkManager; changes need sudo)

```
nmcli dev status ← every device and the profile it uses (see below)
nmcli con show          nmcli con show eno1 ← all profiles / every setting of one
nmcli con up eno1       nmcli con down eno1 ← start (also applies edits) / stop
nmcli con mod eno1 ipv4.method manual ipv4.addresses 192.168.1.50/24 ← static IP (method auto = DHCP)
nmcli con mod eno1 ipv4.gateway 192.168.1.1 ipv4.dns "192.168.1.1 1.1.1.1" ← apply: con up
# bridge main with eno1 as its port
nmcli con add type bridge ifname main con-name main bridge.stp no \
    ipv4.method manual ipv4.addresses 192.168.1.50/24 ipv4.gateway 192.168.1.1
nmcli con add type bridge-slave ifname eno1 master main ← plug eno1 in
nmcli con up main ← then delete eno1's old profile
```

Reading nmcli dev status

DEVICE	TYPE	STATE	CONNECTION
main	bridge	connected	main
eno1	ethernet	connected	bridge-slave-eno1
eno2	ethernet	unavailable	--
wlp2s0	wifi	disconnected	--
virbr0	bridge	unmanaged	--

unavailable = no cable or link · **disconnected** = usable, no profile active · **unmanaged** = NM ignores it (another tool owns it)

Debian / Proxmox: ifupdown

Config in **/etc/network/interfaces**:

```
auto main
iface main inet static ← or: inet dhcp
    address 192.168.1.50/24
    gateway 192.168.1.1
    bridge-ports eno1
    bridge-stp off
```

- Apply: **ifreload -a** (ifupdown2, Proxmox) or **systemctl restart networking**.

Remember: over SSH use **sudo netplan try**: if you get cut off, it puts the old config back by itself. The IP goes on the bridge **main**, never on **eno1** as well.